From Pixels to a Periodic Phase Signal: Per-Shot Kinematic Inference and Its Effect on (A_s, A_c) Recovery

Contents

1 Problem Statement

A gradiometer measures a weak, periodic phase signal $\beta = (A_s, A_c)$ by repeating a shot $i = 0, \ldots, N-1$ at a fixed rate, injecting a known sinusoidal phase modulation

$$\delta\phi_i(\beta) = A_s \sin(2\pi f i) + A_c \cos(2\pi f i) \tag{1}$$

between two atom-interferometer (AI) measurement planes ($z = 0$ and $z = 100$), and reading out the result as a pixel-resolved fluorescence image at each plane. Each shot also carries an independent, a priori unknown common-mode phase offset $\phi_i \sim \text{Uniform}(0, 2\pi)$ (laser phase noise, not part of the signal), and the atom cloud itself has a shot-dependent 8-dimensional phase-space state θ_{iz} (mean position/velocity and Gaussian spread, at plane z) that is *not* directly observed — only its effect on the recorded image is.

The question this note answers empirically: **how much does it matter, for recovering β, how well θ_{iz} is estimated from the image?** We compare four ways of getting θ_{iz} (from a null guess, to a prior-art moment estimator, to the best pixel-likelihood estimator developed in this project, to the oracle/true value), holding the downstream signal-recovery step fixed, at two photon-count regimes (10^6 and 10^8 atoms per shot). No analytic/Fisher-information arguments are used anywhere in this note — every number reported is a direct, empirical fit-vs.-ground-truth comparison on simulated data with known truth.

2 The Reduction: From (A_s, A_c) to Per-Shot θ_{iz}

This section derives why estimating β reduces to (1) a per-shot, per-plane estimate of θ_{iz} from the image, followed by (2) a signal-fit step that takes those estimates (and per-shot photon counts) as input. The notation matches `notes/joint_profile_inference.tex` ("Per-Shot Pixel Likelihood" section there) exactly.

Scope of the reduction, stated up front. Two facts below are *exact* regardless of how the image is read out: the per-shot phase ϕ_i is a single scalar nuisance shared across both planes (§2.2), and β enters *only* through the scalar $\delta\phi_i(\beta)$ (§2.3). Two further steps are *approximations*, and it is important not to conflate them with the exact structure:

1. **Plug-in θ.** Eq. 4 fixes θ at a per-shot point estimate $\hat{\theta}_{iz}$ and maximises over β, rather than marginalising θ against its posterior. This is exact only in the sharp-θ-posterior (high photon-count) limit; in general it discards the (β-dependent) width of the θ-posterior.
2. **Position averaging.** As *implemented* (`map_inference.py`'s `total_logL_fast` / `_ai_ll_batch`), the signal-fit step uses only the *port-summed total counts* (n_g, n_e) per plane against the *spatially-integrated* coherence coefficients (A, C^c, C^s) — i.e. a two-bin (port-g/port-e) Poisson likelihood per plane, not the full pixel-resolved image likelihood. See §2.5 for why this is *not* the same as the full-image likelihood, and $\hat{\theta}$ is *not* a sufficient statistic for β.

In other words, the clean statement "β-fit needs only $\hat{\theta}$ and counts, not the images" is a property of the *position-averaged* (port-population) model that this pipeline actually uses — it is not a lossless reduction of the full-image likelihood.

2.1 The observation model

At AI z, pixel b, detection state $s \in \{g, e\}$, shot i:

$$n_{izsb} \sim \text{Poisson}\big(\lambda_{izsb}\big), \qquad \lambda_{izsb} = L_{iz}\Big[A_{sb}(\theta_{iz}) + C^c_{sb}(\theta_{iz}) \cos\Phi_{iz} + C^s_{sb}(\theta_{iz}) \sin\Phi_{iz}\Big], \tag{2}$$

where $\Phi_{iz} = \phi_i + \delta\phi_i(\beta) \cdot [z{=}100]$ is the total phase seen at plane z, (A, C^c, C^s) are the PSMAP-model coherence coefficients (Gaussian-cloud-averaged atom optics response, a deterministic function of θ_{iz} only – see the "Pixel ACS" section of `joint_profile_inference.tex`), and L_{iz} is a total-count normalisation fixed by the data (same reference, "Objective Rescaling" recap equation), not a free parameter.

2.2 Step 1: the nuisance phase is a per-shot, per-plane-pair quantity

ϕ_i appears *identically* in both planes' Φ_{iz} (shifted by the known $\delta\phi_i(\beta)$ at $z{=}100$) – it is a single scalar nuisance per shot, not per plane. Marginalising it out of the per-shot likelihood,

$$p(n_{i,0}, n_{i,100} \mid \theta_{i,0}, \theta_{i,100}, \beta) = \int_0^{2\pi} p(n_{i,0} \mid \theta_{i,0}, \phi_i)\, p(n_{i,100} \mid \theta_{i,100}, \phi_i + \delta\phi_i(\beta))\, \frac{d\phi_i}{2\pi}, \qquad (3)$$

gives the shot's *marginal* log-likelihood as a function of $(\theta_{i,0}, \theta_{i,100}, \beta)$ alone – ϕ_i never needs to be reported or stored once this integral is done. This is a 1-dimensional, periodic, and (empirically, this project) cheap-to-integrate-numerically nuisance – see §3.3 for how it is done in practice.

2.3 Step 2: β enters *only* through $\delta\phi_i(\beta)$, which is known given θ

For *fixed* $\theta_{i,0}, \theta_{i,100}$, the marginal likelihood (Eq. 3) is a function of β *only* through the scalar $\delta\phi_i(\beta)$ – a known, linear-in-β function of the shot index i (Eq. 1). This is the entire mechanism by which repeated shots pin down (A_s, A_c): each shot supplies one (generally weak, individually) constraint on $\delta\phi_i(\beta)$, and because $\delta\phi_i(\beta)$ traces out a known sinusoid in i, combining N shots' constraints via

$$\hat{\beta} = \arg\max_{\beta} \sum_{i=0}^{N-1} \log p\big(n_{i,0}, n_{i,100} \mid \hat{\theta}_{i,0}, \hat{\theta}_{i,100}, \beta\big) \qquad (4)$$

recovers β with a precision that improves with N (standard harmonic regression). The entire image-processing burden is front-loaded into estimating θ_{iz} per shot, and every method compared in this note differs *only* in how that front-loaded estimate is produced. What information the signal-fit likelihood in Eq. 4 actually consumes – the full pixel-resolved image, or a position-averaged summary of it – is the subject of §2.5, and matters for how lossless the reduction is.

2.4 The reduction, stated plainly

> *Estimating the periodic signal (A_s, A_c) from N shots of pixel-resolved images reduces to: (a) for each shot and each plane, estimate the 8-dimensional cloud state θ_{iz} from that single image (after marginalising the shot's own unknown common phase ϕ_i); then (b) fit β against the known sinusoidal structure of $\delta\phi_i(\beta)$, using only the per-shot $\hat{\theta}_{iz}$ and raw photon counts as input.*

This is why the four methods compared below (§3) differ *only* in step (a) – step (b) (Eq. 4, implemented via `map_inference.py`'s `batch_acs_gh` + `total_logL_fast` pipeline) is held *identical* across all four, so any difference in the resulting $\hat{\beta}$ is attributable entirely to the quality of the per-shot kinematic estimate.

2.5 What step (b) actually consumes: a port-population likelihood, not the full image

The reduction quoted above is exact as a *structural* statement (Steps 1–2), but the phrase "uses only $\hat{\theta}$ and raw counts, not the images" hides a modelling choice that is worth stating precisely, because it makes the reduction lossy at the full-image level.

What is implemented. The signal-fit likelihood (`total_logL_fast` → `_ai_ll_batch`) evaluates, per shot and per plane, a *two-bin Poisson* likelihood in the total port counts $(n_g, n_e) = \big(\sum_b n_{izgb},\ \sum_b n_{izeb}\big)$ against *spatially-integrated* coefficients $\big(A_s, C_s^c, C_s^s\big) = \sum_b \big(A_{sb}, C_{sb}^c, C_{sb}^s\big)$ (the six numbers returned by `batch_acs_gh` per shot). The pixel-resolved image n_{izsb} enters the pipeline *only* through the upstream estimate $\hat{\theta}_{iz}$; it never appears in the β-fit itself. This is a *position-averaged* (port-population) likelihood.

Why this is not the full-image likelihood. In the observation model (Eq. 2) the phase enters through *pixel-dependent* fringe coefficients $C_{sb}^c(\theta)$, $C_{sb}^s(\theta)$: the signal phase Φ_{iz} (hence $\delta\phi_i(\beta)$, hence β) modulates the *spatial pattern*, not merely the per-port totals. Summing over pixels to form (n_g, n_e) keeps only the projection $\sum_b C_{sb}^c$ (one number per port) and discards the rest of the fringe's β-dependence. Consequently:

- $\hat{\theta}_{iz}$ is **not** a sufficient statistic for β: there is no factorisation theorem here, and indeed $\hat{\theta}$ is estimated *after* marginalising ϕ_i (Eq. 3), so by construction it carries essentially no phase information. The β-information and the θ-information live in different features of the image.
- The full-image profile likelihood $\prod_i p\big(\{n_{izsb}\} \mid \hat{\theta}_{iz}, \beta\big)$ — i.e. Eq. 4 taken literally, with the full pixel image rather than the port totals — would additionally exploit the pixel-resolved fringe, and is therefore (weakly) *more* informative about β than the port-population fit this note uses. How much is left on the table depends on how strongly C_{sb}^c, C_{sb}^s vary across pixels relative to their integrals; it is not quantified here.

Where the reduction *is* clean. For the position-averaged model that is actually run, the statement "estimate θ from the image, then fit β from the port counts" is self-consistent and (up to the plug-in-θ approximation of §2) exact: the port populations depend on β only through $\delta\phi_i(\beta)$ and the θ-dependent integrated coefficients, exactly as Eq. 4 uses them. The caveat is only that this port-population likelihood is a deliberately averaged view of the data, so the comparison in this note is between kinematic estimators *holding that averaged signal-fit fixed* — it does not claim to have extracted every bit of β-information the full images contain.

3 Methods Compared

All four methods below take one shot's pixel image (at one plane z) and return an estimate $\hat{\theta}_{iz} \in \mathbb{R}^8$; nothing else in the pipeline changes between them.

3.1 Null (uninformative baseline)

$$\hat{\theta}_{iz}^{\text{null}} = \bar{\theta} \quad \text{(the prior mean, every shot, always).}$$

Uses *no* image information whatsoever. This is the floor any real method must beat, and (since it never varies) it directly shows what $\hat{\beta}$ looks like when the signal-fit step (Eq. 4) is handed a fixed, uninformative, shot-independent guess for θ at every shot – isolating how much of the ultimately-recovered β precision, if any, survives with zero per-shot kinematic information.

3.2 MAP-moments (prior art, `map_inference.py`)

The existing pipeline: reduce the image to its first two port-summed spatial moments $(\hat{\mu}_{xf}, \hat{\mu}_{yf}, \hat{V}_{xf}, \hat{V}_{yf})$, then apply the closed-form Kalman-gain MAP update $\hat{\eta} = \bar{\eta} + K(\hat{M} - A\bar{\eta})$ (Eq. in `map_inference.py`'s `map_eta`) mapping the 4 observed moments to the 10-dimensional free-flight state η (position/velocity means + full covariance, including a cross term C_{xv0} that θ does not have). For

comparison against the other three methods' θ representation, η is projected down to θ by taking $(\mu_{x0}, \mu_{y0}, \mu_{vx0}, \mu_{vy0})$ directly and $\sigma_\bullet = \sqrt{V_\bullet}$ for the four spread components, discarding the estimated cross-covariance terms (which θ-based methods do not attempt to estimate at all, and which the true generative model sets to zero exactly). This is a MAP point estimate under a Gaussian approximation to the moment likelihood – it explicitly does *not* use any pixel structure beyond the first two spatial moments.

3.3 Pixel-likelihood, best configuration

The method developed over the course of this investigation: marginalise ϕ_i by *direct numerical quadrature* over a dense grid ($M = 2000$ points, verified converged to $\sim 10^{-9}$ absolute by varying M from 500 to 8000 – §A.2), then optimise θ_{iz} (8-dimensional) against the resulting smooth marginal log-likelihood via plain L-BFGS-B (`scipy.optimize.minimize`), with the model's own per-ϕ-grid-point analytic-ish gradient (from `pixel_acs_grad.pixel_acs_and_grad`) chained through the log-sum-exp marginalisation to give an efficient gradient for the outer θ-optimisation (§A.5). This inverts the naive computational order (fix ϕ, optimise θ conditionally at every grid point) that was found, over an extended debugging effort, to *not* converge reliably at 10^8 atoms – see `notes/joint_profile_inference.tex`'s "2026-07-20" section for the full negative result this reframing supersedes.

Best configuration, established empirically (§A.4):

- `bins=32` (pixel grid resolution for the image binning)
- *tightened* image range: ± 1.54 mm instead of the dataset's native ± 3 mm, set by the 99.99%-flux-containment radius measured across all 40 available runs (§A.3) – essentially free (same bin count, same evaluation cost), and found to give a comparable improvement to doubling the bin count, but at $\sim 1/4$ the compute cost.
- `pixel_n_gh=8` (2D Gauss–Hermite velocity-quadrature order; verified converged, no measurable effect from 4 to 16).

3.4 Oracle

$$\hat{\theta}_{iz}^{\text{oracle}} = \theta_{iz}^{\text{true}} \quad \text{(read directly from simulation metadata)}.$$

Represents the shot-noise floor: the best any θ-based pipeline could possibly do, since it has zero kinematic estimation error and only the signal-fit step's own Poisson noise remains.

4 Experimental Setup

Quantity	Value
Datasets	10^6 atoms/shot (`R80_N200_A1000000_...`), 10^8 atoms/shot (`R40_N50_A100000000_...`)
Runs per dataset	10 independent runs (different noise realisations of the same β)
Shots per run	50 (matched across both atom counts, apples-to-apples)
Signal frequency	$f = 0.3$ (cycles/shot)
True signal	$A_s = 0.08776$, $A_c = 0.04794$ (same for both datasets)
Prior	$\bar{\theta} = (0, 0, 0, 0, 100, 100, 100, 100)\,\mu$m(/s), $\sigma_\theta = 10\,\mu$m(/s) per component
Best-config bins	32, image half-range 1.54 mm (native: 3.0 mm)
Signal-fit (β) pixel grid	bins=16, native full range, `gh_order=12` (unchanged across all 4 methods)

Full derivations of the hyperparameter choices (`bins=32`, tight range, quadrature order) are given in Appendix A.4–A.2.

5 Results

5.1 Kinematic parameter recovery

Figures 1–2 show the pooled residual distributions (estimate − truth, both AI planes, all 10 runs × 50 shots = 1000 samples per method) for all 8 kinematic parameters, at both atom counts. Table 1 gives the corresponding RMSE.

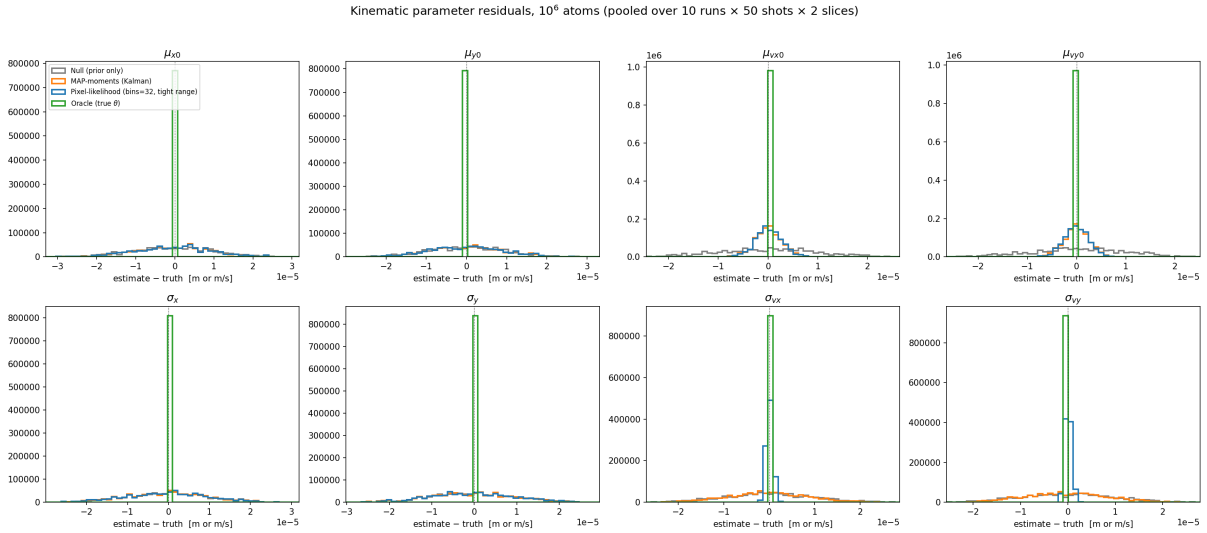

Figure 1: Kinematic parameter residuals, 10^6 atoms.

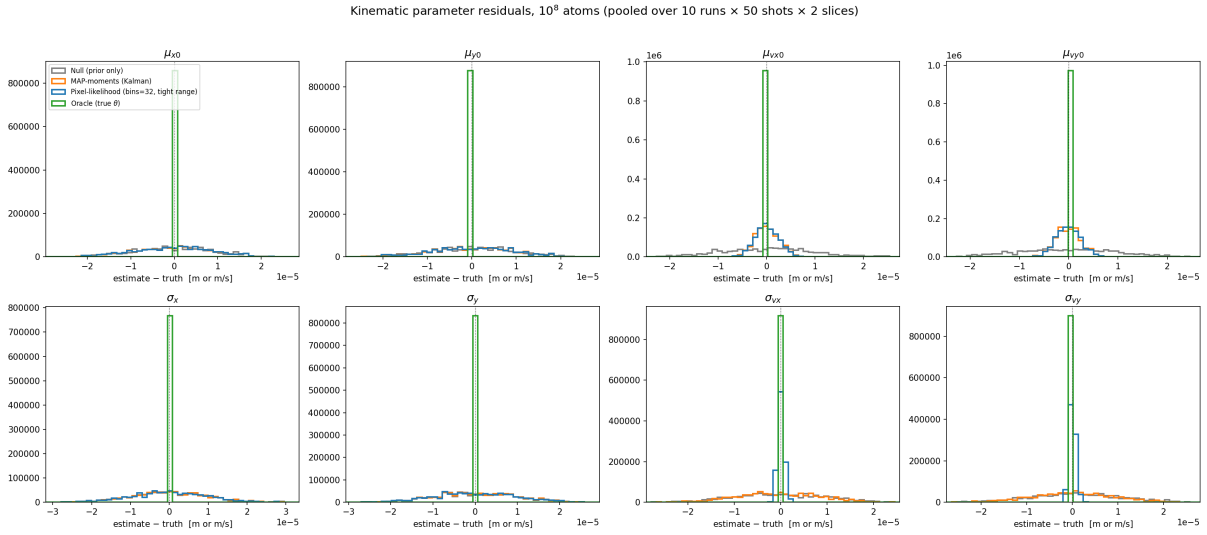

Figure 2: Kinematic parameter residuals, 10^8 atoms.

The pattern is remarkably consistent across the 100× change in atom count: the pixel-likelihood method's advantage over moments is *entirely* concentrated in the two velocity-spread components, at a roughly constant ∼12–13× RMSE reduction regardless of photon budget.

Table 1: Kinematic parameter RMSE by method and atom count, in μm ($\mu_{x0}, \mu_{y0}, \sigma_x, \sigma_y$) or μm/s ($\mu_{vx0}, \mu_{vy0}, \sigma_{vx}, \sigma_{vy}$). Position ($\mu_{x0}, \mu_{y0}$) and position-spread ($\sigma_x, \sigma_y$) are essentially unaffected by which method is used – all three real methods sit close together, well above the (zero, by construction) oracle. Velocity mean (μ_{vx0}, μ_{vy0}) shows a moderate, atom-count-independent advantage for moments/best over null. Velocity spread (σ_{vx}, σ_{vy}) is where the pixel-likelihood method's advantage is concentrated: roughly a 12–13× tighter RMSE than moments, at *both* atom counts.

	10^6 atoms		10^8 atoms	
Parameter	moments	best	moments	best
μ_{x0} [μm]	10.17	10.17	9.28	9.28
μ_{y0} [μm]	9.68	9.68	9.30	9.29
μ_{vx0} [μm/s]	2.70	2.64	2.48	2.40
μ_{vy0} [μm/s]	2.56	2.50	2.47	2.41
σ_x [μm]	10.19	10.19	9.87	9.88
σ_y [μm]	10.00	9.99	9.83	9.82
σ_{vx} [μm/s]	8.92	**0.705**	8.91	**0.670**
σ_{vy} [μm/s]	8.83	**0.691**	8.97	**0.673**

Checking the mechanism directly, rather than assuming it. A first hypothesis was that this concentrates on velocity spread because the moment method only observes the detection-plane position variance $V_{xf} = V_{x0} + 2TC_{xv0} + T^2V_{vx0}$, a single scalar mixing three unknowns – predicting that moments' σ_x and σ_{vx} *errors* should be anti-correlated (trading off against each other to keep V_{xf} fixed). Figure 3 tests this directly by plotting the joint residual distribution for exactly this pair (and the analogous mean-parameter pair), per method. **The naïve trade-off hypothesis is wrong for moments**: (σ_x, σ_{vx}) residuals are essentially uncorrelated for moments (Pearson $r = -0.06$ at 10^8 atoms, -0.04 at 10^6) – moments' large σ_{vx} error is not a correlated trade-off against σ_x, it is simply weakly constrained and close to independent prior-dominated noise. What *is* true, and turns out to be the more informative finding: the pixel-likelihood method's (σ_x, σ_{vx}) residuals are almost perfectly anti-correlated ($r = -0.99$, both atom counts) – and the same near-perfect anti-correlation appears for the mean pair (μ_{x0}, μ_{vx0}) for *both* methods ($r = -0.97$ moments, $r{\approx}-1.00$ best). This is the signature of a genuinely well-constrained detection-plane observable (approximately $\mu_{xf} \approx \mu_{x0} + T\mu_{vx0}$, and analogously for the variance): a good estimator's errors in the two initial-state components must move oppositely to keep the well-determined combination correct, concentrating residual uncertainty in the orthogonal, genuinely underdetermined direction. Moments shows this structure clearly for the *mean* pair but not the *spread* pair; the pixel-likelihood method shows it clearly for *both* – i.e., the moment method fails to extract the (nonlinear, quadratic-in-cloud-parameters) V_{xf} constraint at all, while the pixel likelihood recovers it about as well as it recovers the (linear) μ_{xf} constraint. This is a more precise, and different, explanation than the originally-hypothesized simple trade-off, and was only established by checking the correlation structure directly rather than assuming it from the model equations.

5.2 Signal (A_s, A_c) recovery

Figure 4 shows $(\hat{A}_s, \hat{A}_c)$ for all 10 independent runs, by method and atom count. Table 2 gives the RMSE.

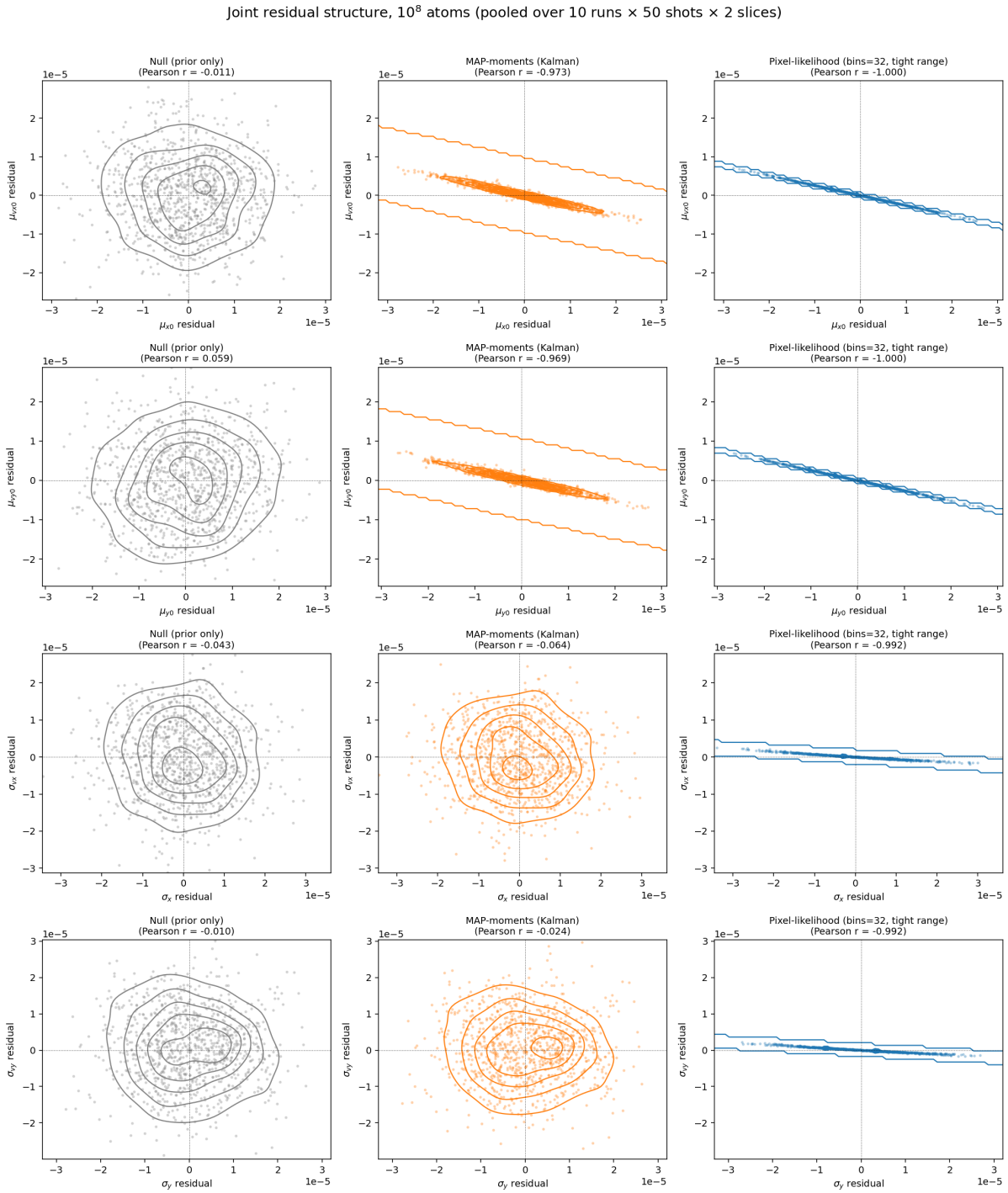

Figure 3: Joint residual structure for physically-coupled parameter pairs, 10^8 atoms (see Appendix A.1 for the 10^6-atom version, which shows the identical pattern). Density contours from a Gaussian KDE; points shown at 25% opacity. Note the near-perfect anti-correlation for *both* pairs for the pixel-likelihood method, vs. only the mean pair for moments.

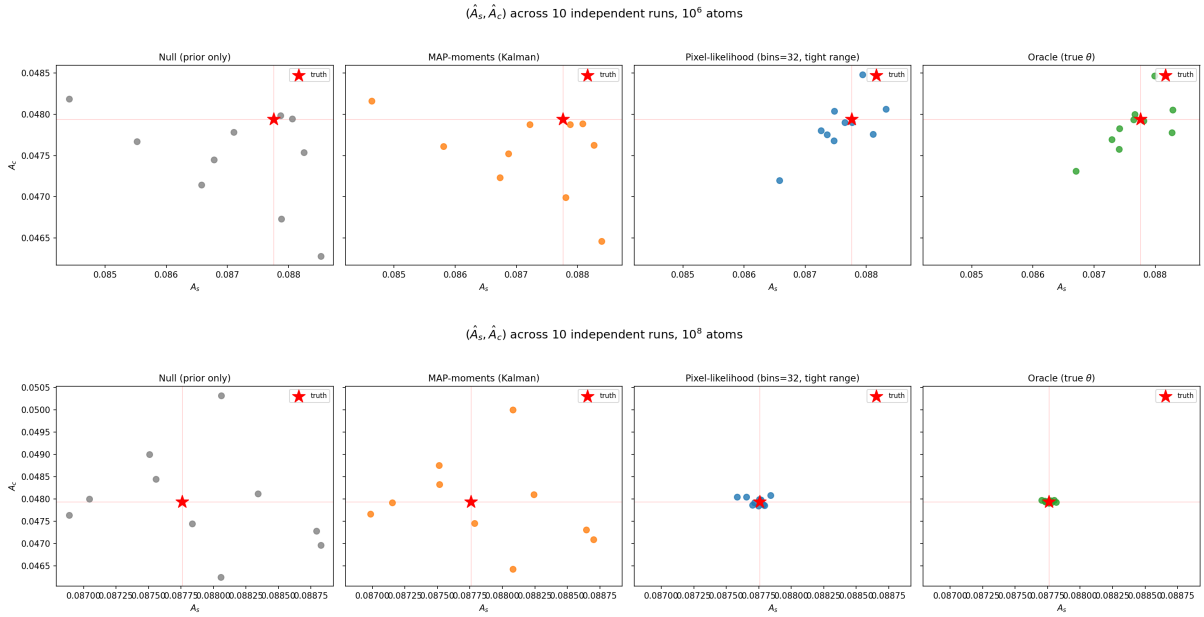

Figure 4: $(\hat{A}_s, \hat{A}_c)$ across 10 independent runs. Top: 10^6 atoms. Bottom: 10^8 atoms. Note: axes are shared across the 4 panels within each row, so the visual tightening from null/moments to best/oracle is directly comparable.

Table 2: Signal (A_s, A_c) RMSE by method and atom count, in mrad (A_s, A_c are phase-modulation amplitudes: $\delta\phi_i(\beta) = A_s \sin(\cdot) + A_c \cos(\cdot)$ is a phase, Eq. 1, so A_s, A_c carry the same units), $N = 10$ independent runs.

	10^6 atoms		10^8 atoms	
Method	RMSE(A_s) [mrad]	RMSE(A_c) [mrad]	RMSE(A_s) [mrad]	RMSE(A_c) [mrad]
Null	1.412	0.737	0.641	1.078
MAP-moments	1.284	0.636	0.566	0.936
Pixel-likelihood (best)	**0.495**	**0.321**	**0.074**	**0.086**
Oracle	0.468	0.304	0.032	0.020
Best / moments ratio	0.386	0.505	0.131	0.092
Best / oracle ratio	1.058	1.056	2.31	4.30

6 Discussion

The kinematic-recovery advantage survives into a real, substantial β-level improvement, and grows sharply with photon count. At 10^6 atoms, the pixel-likelihood method beats the existing moment-based estimator by 2.0–2.6× in β RMSE, and lands within $\sim 6\%$ of the oracle (shot-noise) floor – there is very little room left to improve at this photon budget, consistent with earlier work in this project finding `map_inference.py` already close to its Cramér–Rao bound at 10^6 atoms. At 10^8 atoms the picture changes qualitatively: moments barely beats the uninformative null baseline (RMSE ratio $0.566/0.641 = 0.88$ on A_s), while the pixel-likelihood method is 7.6–10.9× better than moments and still has a genuine 2.3–4.3× gap remaining to the oracle floor – i.e., at high photon count there is both a large amount of information the moment method fails to extract, *and* a large amount the pixel-likelihood method (in its current, non-exhaustively-tuned configuration) also leaves on the table.

Why the effect is atom-count-dependent even though the kinematic RMSE ratio (Table 1) is not: the $\sim$12–13× tightening of σ_{vx}, σ_{vy} is essentially constant across the 100× atom-count range, yet its effect on β grows by roughly 4–5× from 10^6 to 10^8. This means the downstream signal-fit step's *sensitivity* to residual velocity-spread error, not the size of that error itself, is what scales with photon count – plausible since more photons per shot let the signal-fit likelihood itself resolve finer distinctions in the effective phase information, making it less tolerant of a given absolute kinematic bias. This is stated as an observation, not yet independently verified by a dedicated sensitivity check.

Caveats. $N_{\mathrm{RUNS}} = 10$ is a real but not huge sample – the β RMSE numbers in Table 2 carry meaningful sampling uncertainty of their own (not quantified here; a bootstrap or larger-N rerun would tighten this). The pixel-likelihood configuration (`bins=32`, tight range) was chosen by a hyperparameter search focused on *kinematic*-level RMSE (Appendix A.4), not by directly optimising β-level RMSE – the remaining oracle gap at 10^8 atoms suggests further gains may be available by pushing `bins` past 32 or tuning the range cutoff more aggressively, not yet tested end-to-end through the β-fit at this larger scale. Finally, all "best"-method fits used the identical downstream β-fit configuration (`bins=16`, native range, `gh_order=12`) as the other three methods, by design (§2) – so none of the reported gain comes from also improving that shared step.

A Supporting Methodology

A.1 Joint residual structure at 10^6 atoms

A.2 Quadrature convergence for the ϕ-marginal

The ϕ-marginal integral (Eq. 3) is evaluated on a uniform grid of M points over $[0, 2\pi)$ via `scipy.special.logsumexp` for numerical stability. M was varied from 500 to 8000 (on 3 independent test shots) and the resulting $\hat{\theta}_{\mathrm{MAP}}$ tracked: it is stable to $\sim 10^{-9}$ absolute (against a parameter natural scale of $\sim 10^{-5}$) already at $M = 2000$, which is the value used throughout this note.

A.3 Tight-range derivation

The dataset's native image half-range (±3 mm) is set generously to guarantee no clipping under worst-case cloud excursions, but the *bulk* of the photon flux occupies a much smaller region. Aggregating photon counts vs. |position| across all 40 available runs (both 10^6 and 10^8 atom datasets separately, giving consistent results: 1.540 mm vs. 1.537 mm respectively) gives the following flux-containment radii:

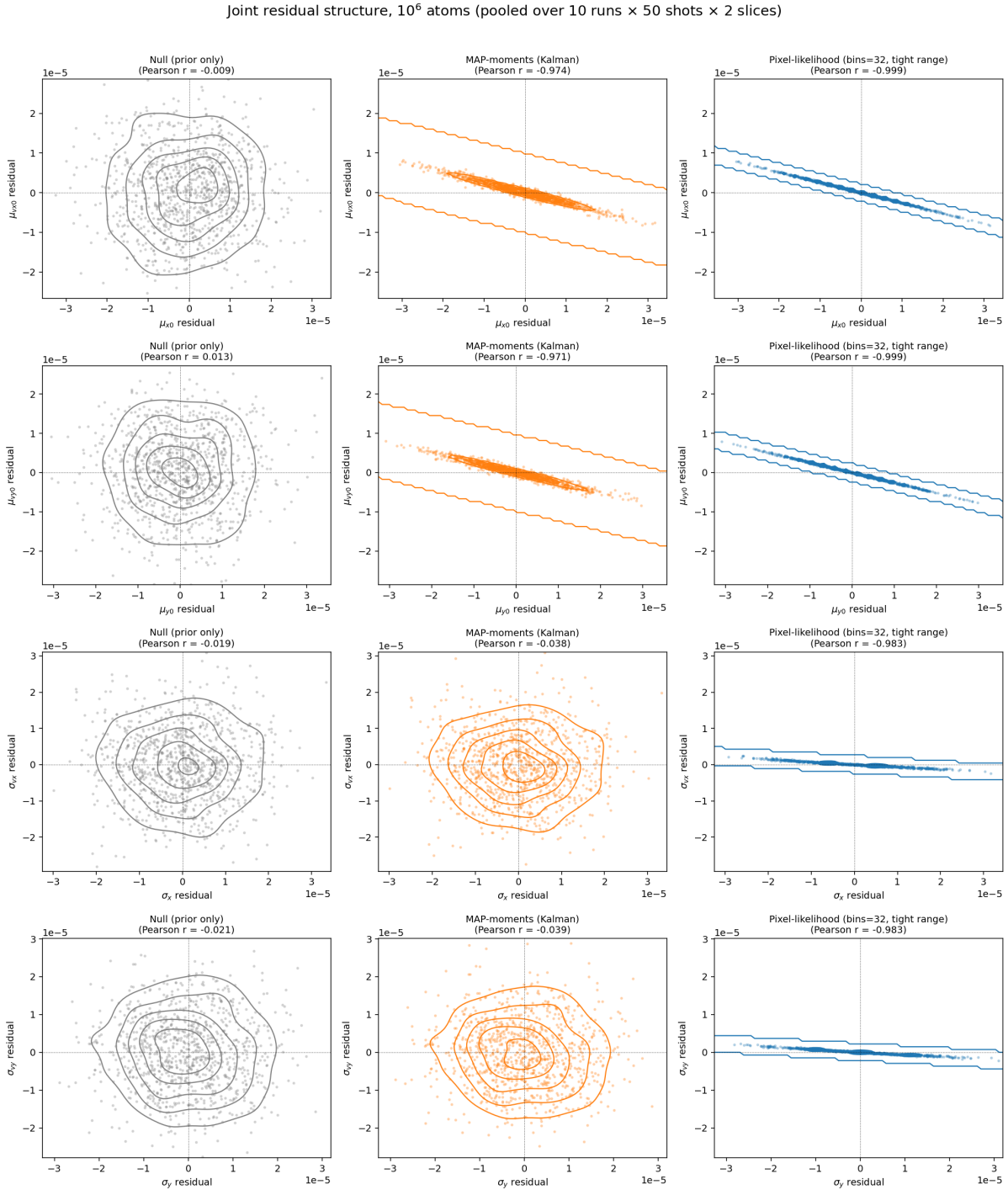

Figure 5: Joint residual structure (same pairs, same layout as Figure 3), 10^6 atoms. The correlation pattern discussed in §5 is essentially identical to the 10^8-atom case.

Fraction of total counts contained	half-range needed	ratio to native
99%	1.003 mm	0.335
99.9%	1.294 mm	0.431
99.99%	1.540 mm	0.513
99.999%	1.762 mm	0.587

99.99% containment (1.54 mm) was chosen: it loses only 1 photon in 10^4 (negligible relative to Poisson noise itself) while roughly halving the pixel size at fixed `bins` – i.e., a resolution improvement essentially free of extra compute, since the number of bins (and hence the model-evaluation cost) is unchanged; only the physical size each bin covers shrinks.

A.4 Hyperparameter sweep: bins vs. tight range, and whether they compound

Comparing `bins`$\in \{16, 32\}$ crossed with {native range, tight range} on the same 50-shot test set (velocity-spread RMSE, the parameters where any effect is seen at all):

Configuration	RMSE(σ_{vx}) [μm/s]	RMSE(σ_{vy}) [μm/s]
bins=16, native range (baseline)	0.869	0.860
bins=32, native range	0.714	0.663
bins=16, tight range	0.725	0.681
bins=32, tight range (both)	0.699	0.634

The two do *not* compound multiplicatively: combining both tricks (0.805/0.737× baseline) is only marginally better than either alone (0.821/0.771× for bins=32; 0.835/0.792× for tight range), far short of the $\sim$0.63–0.68× a naive independent-effects multiplication would predict. This indicates bins and range are tapping a shared underlying resource (spatial resolution near the flux-dense region), not two independent information sources, and that a real ceiling exists around $\sim$20–26% improvement from spatial resolution alone at this photon range. `bins=32` + tight range was used as "best" throughout this note as the best-available combination, despite the modest marginal gain over tight-range-alone, for consistency with the rest of the investigation; a cost-sensitive deployment might prefer `bins=16` + tight range for $\sim 4\times$ lower compute at comparable quality.

A third bins value reveals the tight-range trick is not universally beneficial. Adding `bins=8` + tight range to the comparison:

Configuration	RMSE(σ_{vx}) [μm/s]	RMSE(σ_{vy}) [μm/s]
bins=16, native range (baseline)	0.869	0.860
bins=8, tight range	**0.955**	**0.962**
bins=16, tight range	0.725	0.681
bins=32, tight range	0.699	0.634

At `bins=8`, tight range is *worse* than the bins=16, native-range baseline (1.10×/1.12×, i.e. a $\sim$10% regression) – despite each pixel still being physically smaller in absolute terms (0.385 mm vs. the native-range bins=8 pixel width of 0.75 mm). This means the earlier framing ("tight range is essentially free and helps") is incomplete: it only helps once `bins` is already large enough to exploit the freed-up resolution; at too few bins, whatever information is lost by clipping the (mostly empty, but not perfectly empty) outer image apparently outweighs the gain from finer pixel spacing. The bins=16→32 trend genuinely flattens (a real, near-plateau ceiling, §A.4 above), but bins=8 is not on that same monotonic curve – it is a qualitatively worse

regime, not merely a smaller improvement. Any future attempt to push `bins` *below* 16 with the tight-range trick should not be assumed safe without re-checking this non-monotonicity.

A.5 Semi-analytic gradient for the ϕ-marginal, and why it was necessary

An initial attempt to estimate θ's local posterior curvature (Hessian, for a Laplace-approximation uncertainty estimate) via finite differences of the raw marginal NLL failed badly: the resulting Hessian had negative eigenvalues and several exactly-zero implied variances, for a majority of test shots, regardless of step-size tuning (fixed or adaptive). The root cause: *differentiating an already-approximate derivative* – `pixel_acs_grad.pixel_acs_and_grad` itself computes θ's gradient via an internal finite difference (`h_theta=1e-9`), and a second (outer) finite difference on top of that compounds numerical noise in a way that step-size tuning could not reliably fix.

The fix: since ϕ only enters the model by combining θ-dependent, ϕ-independent coefficients (A, C^c, C^s) via $\cos\Phi, \sin\Phi$ (Eq. 2), `pixel_acs_and_grad` needs to be called only *once* per θ (already the case), and its returned per-θ-component gradient can be chained through the log-sum-exp marginalisation analytically:

$$\frac{\partial\,\text{marginal NLL}}{\partial\theta_k} = \sum_{m=1}^{M} w_m \frac{\partial\,\text{NLL}(\theta,\phi_m)}{\partial\theta_k}, \qquad w_m = \text{softmax}_m\big(-\text{NLL}(\theta,\phi_m)\big), \tag{5}$$

i.e. a ϕ-posterior-weighted average of the per-grid-point gradient – no new finite differences introduced at the marginalisation step. This was validated against a plain finite-difference gradient of the marginal NLL at a fixed test point: relative agreement $\sim 10^{-7}$–10^{-4} component-wise, confirming correctness of the chain rule. Using this gradient (`jac=True` in `scipy.optimize.minimize`) also substantially speeds up the θ-optimisation itself (converges in ~ 6 iterations vs. needing scipy's own finite-difference outer gradient otherwise).